

TEMA 3. Excepciones

1. Empecemos un tema sobre control de errores en lenguajes de programación, con algo básico. En C, donde no existen las excepciones, pongamos un ejemplo de una raíz que toma número flotante positivo. Queremos controlar el error si la función recibe un número negativo. El usuario debe ser informado pero desde fuera de la función **raiz** ¿Cómo indicamos ese error?. Enumera dos opciones diferentes de diseñar, poniendo un ejemplo de código de cada una.

☑ 1. Opción 1: Usar un valor especial de retorno + una variable externa para indicar el error

En C es típico devolver un valor "sentinela" (por ejemplo **-1.0**) y que la función establezca una **variable global** o **una variable pasada por referencia** que indique si hubo error.

✓ Ejemplo en C usando un parámetro por referencia para señalar el error

```
#include <stdio.h>
#include <math.h>

double raiz(double x, int *error) {
    if (x < 0) {
        *error = 1;        // indicamos error
        return -1.0;      // valor sentinela
    }
    *error = 0;            // sin error
    return sqrt(x);
}

int main() {
    int err;
    double r = raiz(-9, &err);

    if (err) {
        printf("Error: número negativo\n");
    } else {
        printf("Resultado: %f\n", r);
    }
}
```

Ventaja: sencillo.

Desventaja: el valor devuelto puede confundirse con un resultado válido.

☑ Opción 2: Devolver el resultado por referencia y usar el valor de retorno como código de error

Aquí **la función no devuelve el resultado directamente**, sino que escribe el resultado en una variable que recibe por referencia.
El valor de retorno (`int`) se usa para indicar si hubo error o no.

✓ Ejemplo en C devolviendo código de error

```
#include <stdio.h>
#include <math.h>

int raiz(double x, double *resultado) {
    if (x < 0) {
        return -1;    // código de error
    }

    *resultado = sqrt(x);
    return 0;        // éxito
}

int main() {
    double r;
    int code = raiz(-9, &r);

    if (code != 0) {
        printf("Error: número negativo\n");
    } else {
        printf("Resultado: %f\n", r);
    }
}
```

- Ventaja:** los códigos de error son claros y no hay ambigüedad sobre el valor del resultado.
Desventaja: obliga a trabajar siempre con punteros.

Resumen

Opción	Mecanismo	Qué devuelve la función	Cómo avisa del error
1. Valor especial + flag externo	Devuelve resultado o valor sentinela	<code>double</code>	Parámetro <code>error</code>
2. Código de error + resultado por referencia	Devuelve estado	<code>int</code>	Valor de retorno

2.1 ¿Qué es una *excepción*?

Una **excepción** es un mecanismo que permite **detectar y gestionar situaciones anómalas** (errores) que ocurren durante la ejecución de un programa.

Cuando una operación no puede completarse correctamente (por ejemplo, dividir entre cero, acceder fuera de un array o intentar abrir un fichero inexistente), se **lanza una excepción**.

La excepción **interrumpe el flujo normal del programa** y transfiere el control a un bloque preparado para manejar ese problema.

2.2 🌀 ¿Para qué las usa un programador?

Los programadores utilizan las excepciones para lograr dos objetivos fundamentales:

✓ 1. Cuando implementa funciones

Para **informar automáticamente al exterior** de que ocurrió un error, sin depender de valores especiales de retorno (como en C).

Esto permite separar la *lógica normal* de la *lógica de error*.

Ejemplo en Java (función que lanza una excepción)

```
public static double raiz(double x) throws IllegalArgumentException {  
    if (x < 0) {  
        throw new IllegalArgumentException("El número no puede ser negativo.");  
    }  
    return Math.sqrt(x);  
}
```

✓ 2. Cuando llama a funciones

Para poder **manejar el error de manera controlada**, evitando que el programa se detenga abruptamente.

Ejemplo en Java (manejo de la excepción)

```
try {  
    double r = raiz(-9);  
    System.out.println("Resultado: " + r);  
} catch (IllegalArgumentException e) {  
    System.out.println("Error al calcular: " + e.getMessage());  
}
```

De esta manera, el programador:

- Evita que el programa falle inesperadamente.

- Puede mostrar mensajes útiles al usuario.
- Mantiene el código más limpio y ordenado.

Resumen breve

Aspecto	Explicación
Qué es una excepción	Un mecanismo para señalar errores durante la ejecución.
Para qué sirven	Para detener el flujo normal e informar de un problema.
Por qué se usan	Separan la lógica normal de la gestión de errores y permiten manejar fallos de forma segura.

3. Reescribe el mismo ejemplo de raíz, pero en Java, metiendo ese método en una clase **Calculadora** y llama a dicho método desde el método **main**, mostrando cómo se puede controlar desde fuera.

☒ Clase **Calculadora** con el método **raiz**

```
public class Calculadora {  
  
    public static double raiz(double x) throws IllegalArgumentException {  
        if (x < 0) {  
            throw new IllegalArgumentException("El número no puede ser  
negativo.");  
        }  
        return Math.sqrt(x);  
    }  
}
```

☒ Clase principal con **main** llamando a **Calculadora.raiz**

```
public class Main {  
    public static void main(String[] args) {  
  
        try {  
            double resultado = Calculadora.raiz(-9);  
            System.out.println("Resultado: " + resultado);  
        } catch (IllegalArgumentException e) {  
            System.out.println("Error al calcular la raíz: " + e.getMessage());  
        }  
    }  
}
```

```
        System.out.println("Programa terminado.");
    }
}
```

Explicación breve

1. La clase **Calculadora** define el método **raiz**, que:

- Comprueba si el número es negativo.
- En caso de error, **lanza una excepción** (**IllegalArgumentException**).
- Si es válido, devuelve la raíz cuadrada.

2. El método **main**:

- Llama a **Calculadora.raiz(...)**.
- Usa un bloque **try/catch** para **capturar y manejar** la excepción.
- El programa continúa funcionando sin cortarse abruptamente.

4.1 ¿Qué es "*lanzar*" una excepción?

Lanzar una excepción significa **interrumpir el flujo normal del programa** y crear un objeto de tipo excepción que indica que ha ocurrido un error.

En tu método:

```
throw new IllegalArgumentException("El número no puede ser negativo.");
```

Aquí:

- El programa **detiene** la ejecución normal del método **raiz**.
- Se crea una excepción **IllegalArgumentException**.
- Esa excepción "salta" hacia arriba buscando alguien que la controle.

4.2 ¿Qué es "*controlar*" o "*capturar*" una excepción?

Controlar (o capturar) una excepción significa **interceptarla** usando un bloque **try/catch** y decidir qué hacer con ese error (mostrar un mensaje, recuperar el programa, etc.).

Ejemplo desde **main**:

```
try {
    double r = Calculadora.raiz(-9);
    System.out.println("Resultado: " + r);
} catch (IllegalArgumentException e) {
    System.out.println("Error al calcular la raíz: " + e.getMessage());
}
```

Ahí el **catch** **controla** la excepción lanzada por **raiz**.

4.3 ¿Qué es que una excepción se "*propage*"?

Cuando un método **lanza** una excepción y **no la captura**, la excepción se envía automáticamente al **método que lo llamó**.

Si ese método tampoco la captura, sigue subiendo por la **pila de llamadas**, y así sucesivamente, hasta que:

- alguien la capture, o
- llegue al método **main** → si tampoco la captura, el programa termina con error.

4.4 ¿Qué ocurre con las funciones en la **pila de llamadas** durante la propagación?

Imagina este flujo:

```
main → A → B → Calculadora.raiz
```

Y **raiz(-9)** lanza una excepción.

Lo que ocurre es:

1. **raiz se interrumpe** inmediatamente (no sigue ejecutando nada después del **throw**).
2. La excepción sube a **B**.
 - Si B no tiene **try/catch** → **se termina B** automáticamente.
3. La excepción sube a **A**.
 - Si A no tiene **try/catch** → **se termina A**.
4. Llega a **main**.
 - Si **main** tiene un **catch**, la controla.
 - Si no lo tiene, el programa finaliza con error.

👉 **Cada función que no controla la excepción queda abortada**, como si nunca hubiera terminado normalmente.

4.5 ☹ ¿Las funciones que no controlan la excepción se reanudan después?

No. Nunca.

Una vez que una función se ve interrumpida por una excepción y **no la captura**, esa función:

- **Termina abruptamente.**
- **No vuelve a continuar** desde donde se lanzó la excepción.
- **No ejecuta código que viniera después del punto del error** (aunque tuviera más instrucciones).

La única parte que sí se garantiza que se ejecuta es un posible bloque **finally**.

🔗 Ejemplo completo con flujo explicado

```
public class Calculadora {  
  
    public static double raiz(double x) throws IllegalArgumentException {  
        if (x < 0) {  
            throw new IllegalArgumentException("El número no puede ser  
negativo.");  
        }  
        return Math.sqrt(x);  
    }  
}
```

```
public class Main {  
  
    public static void main(String[] args) {  
        System.out.println("Inicio");  
  
        try {  
            double r = Calculadora.raiz(-9); // ← aquí se lanza la excepción  
            System.out.println("Resultado: " + r); // ← ESTO NO SE EJECUTA  
        } catch (IllegalArgumentException e) {  
            System.out.println("Error capturado: " + e.getMessage());  
        }  
  
        System.out.println("Fin del programa");  
    }  
}
```

Flujo real:

1. **main** llama a **raiz(-9)**.

2. **raiz** lanza la excepción.
3. **main** **captura** la excepción.
4. El mensaje del **catch** se muestra.
5. El programa **continúa con normalidad**.

Resumen muy breve (ideal para examen)

- **Lanzar** una excepción → interrumpir el método y notificar un error.
- **Capturar** una excepción → manejar el error con **try/catch**.
- **Propagar** una excepción → la excepción sube por la pila si no se captura.
- Métodos por los que pasa la excepción → **se abortan**, no se reanudan.
- Solo se ejecuta el código que esté en un bloque **finally** si existe.

5. ☒ Ventajas de la *propagación natural* de las excepciones en Java (frente a C)

En C, el programador debe **propagar manualmente** el error devolviendo códigos especiales o usando punteros/variables externas.

En Java, la propagación es **automática**: si un método no captura una excepción, esta sube sola por la pila de llamadas (*stack*).

Esto aporta varias ventajas importantes:

1. Menos código repetitivo

En C, cada función debe comprobar explícitamente códigos de error devueltos por otras funciones:

```
int code = funcionA();  
if (code != 0) return code;
```

Este patrón debe repetirse en todos los niveles.

En Java, si no capturas la excepción, **simplemente se propaga sola**, sin escribir nada.

2. La lógica normal del programa queda más limpia

El código en C mezcla:

- lógica normal

- comprobación de errores
- propagación manual del error

En Java, el método solo se ocupa de su tarea:

```
return Math.sqrt(x);
```

Y si algo falla:

```
throw new IllegalArgumentException();
```

La propagación ya la hace el sistema automáticamente, manteniendo el código **más limpio y legible**.

✓ 3. Es imposible “olvidarse” de pasar el error hacia arriba

En C, un programador puede cometer errores como:

- no comprobar el valor devuelto
- no propagarlo correctamente
- devolver valores incorrectos

En Java, si un método no controla la excepción, el sistema **siempre la propaga automáticamente** por la pila de llamadas.

No hay riesgo de que el error “desaparezca”.

✓ 4. Los métodos intermedios no se ejecutan parcialmente

En C, si no controlas bien los códigos de error, puede ocurrir:

- se ejecuta parte del método
- luego se detecta el error
- pero ya se han modificado datos o recursos

Java, en cambio:

- **interrumpe** el método en el punto exacto del error
- **descarta** automáticamente todos los métodos intermedios
- garantiza que no continúa ningún método que no deba

Esto evita muchos errores lógicos.

✓ 5. La pila se desenrolla automáticamente y se liberan los recursos

En C, liberar recursos (memoria, ficheros...) depende del programador y es fácil olvidarse de hacerlo en caso de error.

En Java:

- la propagación *desenrolla* la pila
- se ejecutan automáticamente los bloques **finally**
- se asegura una limpieza ordenada del estado

✓ 6. Las excepciones llevan información detallada

Una excepción en Java incluye:

- tipo de error
- mensaje
- *stack trace*

Un simple código de error en C no contiene esta información; hay que construirla manualmente.

✓ 7. Simplifica el análisis del flujo del programa

La lógica de "quién maneja el error" es clara:

- Si un método no captura una excepción, **sube automáticamente**.

Esto hace mucho más fácil razonar sobre el comportamiento del programa ante fallos.

🔗 Ejemplo aplicado al caso de **Calculadora.raiz()**

En C, cada función que llamase a **raiz()** tendría que comprobar el error y reenviarlo manualmente.

En Java:

```
public static double raiz(double x) {  
    if (x < 0) {  
        throw new IllegalArgumentException("Número negativo");  
    }  
    return Math.sqrt(x);  
}
```

Si `main()` no la captura, sube.

Si otra función llamó a `main()`, sube más.

Todo ocurre **sin escribir ni una línea extra**.

Resumen breve

La propagación natural de excepciones en Java es mejor que el manejo manual de errores en C porque:

- elimina código repetitivo
 - evita errores humanos
 - limpia y separa la lógica normal del código de error
 - asegura una gestión ordenada de recursos
 - aporta información rica del fallo
 - aborta automáticamente funciones intermedias sin dejarlas en estados inconsistentes
-

6.1 ¿En orientación a objetos, las excepciones suelen ser objetos?

Sí.

En los lenguajes orientados a objetos (como Java), **todas las excepciones son objetos** que heredan de la clase base `Throwable`.

Ejemplo rápido:

```
try {  
    int x = 10 / 0;  
} catch (ArithmeticException e) {  
    System.out.println(e.getClass()); // ← es un objeto real  
}
```

Las excepciones **no son simples códigos numéricos** (como en C), sino instancias de clases con atributos, métodos y herencia.

6.2 ¿Qué ventajas aporta esto en términos de encapsulación?

Al ser objetos, las excepciones:

✓ 1. Pueden encapsular información útil

Una excepción puede incluir:

- un mensaje descriptivo
- la causa de otro error interno
- datos adicionales
- un *stack trace* automático

Todo esto está **empaquetado dentro del propio objeto**, lo cual mejora muchísimo la capacidad de diagnóstico.

```
throw new IllegalArgumentException("Valor fuera de rango");
```

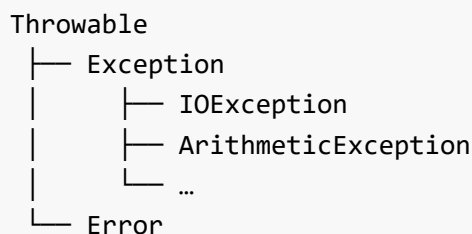
Este objeto guarda:

- tipo de error
- mensaje
- pila de llamadas

En C, toda esta información debería construirse manualmente.

✓ 2. Permiten agrupar errores por jerarquías

Java usa herencia para clasificar las excepciones:



Esto facilita:

- capturar tipos concretos
- o capturar familias enteras de errores
- o diferenciar entre errores recuperables y no recuperables

✓ 3. Se separa claramente la lógica del error

La excepción lleva **su propia información**, no hace falta mezclarla con:

- variables auxiliares
- valores de retorno especiales
- constantes mágicas

La lógica del programa queda más limpia (más encapsulación).

6.3 ¿Podemos crear excepciones personalizadas?

Sí. Por supuesto.

Es una de las grandes ventajas.

En Java, simplemente se crea una clase que herede de `Exception` o `RuntimeException`.

✓ Ejemplo de excepción personalizada en Java

```
public class NumeroNegativoException extends Exception {  
  
    public NumeroNegativoException(String mensaje) {  
        super(mensaje);  
    }  
}
```

Y usarla en tu `Calculadora`:

```
public class Calculadora {  
  
    public static double raiz(double x) throws NumeroNegativoException {  
        if (x < 0) {  
            throw new NumeroNegativoException("El número no puede ser negativo.");  
        }  
        return Math.sqrt(x);  
    }  
}
```

Y controlarla desde `main`:

```
public class Main {  
    public static void main(String[] args) {  
  
        try {  
            double r = Calculadora.raiz(-5);  
            System.out.println("Raíz = " + r);  
        } catch (NumeroNegativoException e) {  
            System.out.println("Error: " + e.getMessage());  
        }  
    }  
}
```



- En POO, **las excepciones son objetos**.
- Esto permite **encapsular** información del error dentro del propio objeto.
- Gracias a esto, el código queda más limpio, claro y orientado a objetos.
- Sí, se pueden crear **excepciones personalizadas** mediante herencia.

7. ¿Qué información esencial lleva *cualquier* objeto excepción en Java?

Todo objeto excepción en Java contiene **al menos tres elementos fundamentales**, que resultan extremadamente útiles cuando llegan al manejador (`catch`):

1. El tipo de excepción (la clase concreta)

Ejemplos:

- `IllegalArgumentException`
- `NullPointerException`
- `IOException`

Esto permite saber **qué tipo exacto de error ha ocurrido**, no solo un número genérico como en C. El tipo ya indica la *naturaleza* del problema.

2. Un mensaje descriptivo del error

Cada excepción puede llevar un mensaje explicando qué ha pasado, por ejemplo:

```
throw new IllegalArgumentException("El número no puede ser negativo");
```

En C tendrías que devolver un código y luego, fuera, convertirlo manualmente a un mensaje. En Java, el objeto excepción *ya encapsula el mensaje*.

3. El *stack trace* (la traza de llamadas)

Este es el elemento clave que marca una gran diferencia frente a C.

El *stack trace* contiene:

- qué métodos estaban en ejecución
- en qué orden
- en qué línea exacta del código ocurrió el error

Por ejemplo:

```
java.lang.IllegalArgumentException: El número no puede ser negativo
    at Calculadora.raiz(Calculadora.java:7)
    at Main.main(Main.java:5)
```

Esto es **oro puro** al depurar y entender el fallo.

Comparación directa con C

Lenguaje	¿Qué información envía una función con error?	¿Ventajas / Desventajas?
C	Un número o código manual (ej. -1)	Muy pobre; no incluye tipo de error ni mensaje ni contexto.
Java	Un objeto excepción con: tipo, mensaje y <i>stack trace</i>	Encapsula toda la información relevante; muy útil para depurar y manejar el error.

Resumen para examen

La información esencial que trae cualquier objeto excepción en Java y que es muy útil al llegar al manejador es:

1. **El tipo de la excepción** (la clase concreta del error).
2. **Un mensaje descriptivo** del problema.
3. **La traza de la pila de llamadas (*stack trace*)**, que indica dónde se produjo el error y cómo llegó hasta allí.

8.1 ☒ ¿Se pueden tener **más de un bloque catch** en Java?

Sí.

Un bloque **try** puede ir seguido de **varios bloques catch**, cada uno destinado a capturar un tipo distinto de excepción.

Ejemplo:

```
try {
    // código que puede lanzar varias excepciones
} catch (IOException e) {
    // manejo específico de IOException
} catch (NumberFormatException e) {
```

```
// manejo específico de NumberFormatException
} catch (Exception e) {
    // manejo genérico
}
```

8.2 ¿Cuántos bloques `catch` se ejecutan?

Solo se ejecuta UNO.

Cuando ocurre una excepción:

1. Java busca el **primer `catch` cuyo tipo coincida** con la excepción lanzada.
2. Ejecuta **únicamente ese `catch`**.
3. Los demás `catch` son **ignorados**.

Nunca se ejecutan dos `catch` para la misma excepción.

Esto se debe a que el manejo de excepciones debe ser **determinista**: una excepción tiene **un único manejador**.

Ejemplo aplicado

```
try {
    double x = Calculadora.raiz(-5);
} catch (NumeroNegativoException e) {
    System.out.println("Error: número negativo");
} catch (Exception e) {
    System.out.println("Otro tipo de error");
}
```

Si `raiz(-5)` lanza `NumeroNegativoException`:

- Se ejecuta **solo** el primer `catch`.
- El segundo `catch` se **ignora**, aunque también podría capturarla (porque una excepción personalizada normalmente hereda de `Exception`).

Resumen breve para examen

-  **Sí**, en Java se pueden tener varios bloques `catch`.
 -  **Se ejecuta solo uno**: el primero cuyo tipo coincida con la excepción lanzada.
 -  La búsqueda es **de arriba hacia abajo**.
-
-

9. ¿Cómo garantizamos que un código se ejecute aunque haya excepciones?

En Java, para garantizar que un bloque de código se ejecute **siempre**, ocurra o no una excepción, se utiliza el bloque **finally**.

El bloque **finally** **siempre se ejecuta**:

- haya o no haya excepción,
- la excepción se capture o no,
- incluso si el método hace **return** dentro del **try** o **catch**.

Su principal uso:

 **cerrar ficheros, liberar memoria, desconectar recursos externos, cerrar sockets, etc.**

Ejemplo 1 — **try + catch + finally**

Este ejemplo captura la excepción, pero **siempre** ejecuta el **finally** para liberar recursos:

```
public class EjemploFinally {  
  
    public static void main(String[] args) {  
  
        try {  
            double r = Calculadora.raiz(-9);    // lanza excepción  
            System.out.println("Resultado: " + r);  
  
        } catch (IllegalArgumentException e) {  
            System.out.println("Error: " + e.getMessage());  
  
        } finally {  
            System.out.println("Cerrando recursos (fichero, conexión...)");  
        }  
  
        System.out.println("Programa continúa.");  
    }  
}
```

✓ Flujo

1. **raiz(-9)** lanza la excepción.
 2. Entra en el **catch**.
 3. Pase lo que pase → **se ejecuta el finally**.
 4. El programa continúa.
-

Ejemplo 2 — **try + finally (sin catch)**

El caso más útil: cuando quieres que la excepción siga propagándose **pero aun así** necesitas liberar recursos.

```
public class EjemploFinallySinCatch {  
  
    public static void main(String[] args) {  
  
        try {  
            double r = Calculadora.raiz(-9);    // lanza excepción  
            System.out.println("Resultado: " + r);  
  
        } finally {  
            System.out.println("Liberando recursos aunque haya error");  
        }  
  
        System.out.println("Esto NO se ejecuta"); // nunca se llega aquí  
    }  
}
```

✓ Flujo

1. `raiz(-9)` lanza excepción.
2. No hay `catch`, así que **la excepción se propaga**.
3. **Pero el `finally` se ejecuta SIEMPRE** antes de que la excepción salga.
4. El método termina abruptamente y no ejecuta nada después del `try`.

Resumen para examen

- El bloque `finally` se ejecuta **siempre**, haya o no excepción.
- Garantiza liberar recursos incluso si la excepción se propaga.
- Puede usarse con `catch` o sin él.
- Sin `catch`, la excepción continúa propagándose después del `finally`.

☒ 10.1. ¿Puede haber un bloque `finally` sin `catch`?

Sí.

En Java, un bloque `try` puede ir seguido de:

- `try + catch`
- `try + catch + finally`
- `try + finally` ← **válido**

Ejemplo válido:

```
try {  
    // código  
} finally {  
    // se ejecuta siempre  
}
```

✓ 10.2. ¿Se ejecuta siempre el bloque `finally`?

Sí, el bloque `finally` se ejecuta siempre, tanto:

- si ocurre una excepción
- como si NO ocurre
- tanto si se captura (`catch`)
- como si se deja propagar

El único objetivo del `finally` es garantizar que **cierto código se ejecute pase lo que pase**, normalmente para:

- cierre de archivos
- liberación de recursos
- desconexión de bases de datos
- desbloques, etc.

✓ 10.3. ¿Y si hay un `return` dentro del `try`?

Incluso si hay un `return`, el bloque `finally` SIEMPRE se ejecuta antes de que el método termine.

Ejemplo:

```
public static int ejemplo() {  
    try {  
        return 10;  
    } finally {  
        System.out.println("Se ejecuta aunque haya return");  
    }  
}
```

Salida:

```
Se ejecuta aunque haya return
```

El método devuelve 10 *después* de ejecutar el `finally`.

Ejemplos completos

✓ Ejemplo con **try + catch + finally**

```
try {
    double r = Calculadora.raiz(-5); // lanza excepción
    return;                          // ni siquiera llegará aquí
} catch (IllegalArgumentException e) {
    System.out.println("Error capturado: " + e.getMessage());
} finally {
    System.out.println("Liberando recursos...");
}
```

Salida:

```
Error capturado: El número no puede ser negativo.
Liberando recursos...
```

✓ Ejemplo con **try + finally** sin **catch**

```
try {
    double r = Calculadora.raiz(-5); // lanza excepción
} finally {
    System.out.println("Esto se ejecuta SIEMPRE");
}

// este código NO se ejecuta, la excepción se propaga
System.out.println("Nunca se llega aquí");
```

Salida:

```
Esto se ejecuta SIEMPRE
Exception in thread "main" ...
```

Resumen breve para examen

- ✓ Sí, el **finally** puede usarse **sin catch**.
- ✓ El **finally siempre se ejecuta**, haya o no excepción.

- ✓ Incluso si hay un **return dentro del try**, el **finally** se ejecuta antes de salir del método.
- ✓ Si no hay **catch**, la excepción **se sigue propagando**, pero *después* de ejecutar el **finally**.

☑ 11.1. ¿Qué son las excepciones *controladas* (checked exceptions)?

Son **excepciones que el compilador obliga a manejar**.

Un método que pueda lanzarlas debe:

- declararlo con **throws**, o
- capturarlas con un bloque **try-catch**.

Estas excepciones **heredan de `Exception`**, pero **NO** de **`RuntimeException`**.

✂ Se usan cuando el error es *esperable, previsible y recuperable*.

Ejemplos típicos (todas checked):

- `IOException`
- `SQLException`
- `FileNotFoundException`
- `ClassNotFoundException`

☑ 11.2. ¿Qué son las excepciones *no controladas* (unchecked exceptions)?

Son excepciones que **NO** es obligatorio declarar ni capturar.

Heradan de **`RuntimeException`**.

Se lanzan normalmente por errores lógicos del programador: índices fuera de rango, accesos nulos, division entre cero...

✂ Se usan cuando el error es *inesperado, difícil de recuperar* o provocado por un *fallo de programación*.

Ejemplos típicos (todas unchecked):

- `NullPointerException`
- `ArithmeticException`
- `IndexOutOfBoundsException`
- `IllegalArgumentException`

☑ 11.3. ¿Qué papel juega **`RuntimeException`**?

`RuntimeException` es la **clase base** de todas las excepciones no controladas.

Toda excepción que herede de `RuntimeException`:

- **no** es obligatorio declararla con `throws`,
- **no** es obligatorio atraparla con `catch`,
- normalmente indica **fallos de programación**, no errores externos.

Ejemplo simplificado:

```
public class MiErrorRuntime extends RuntimeException {
    public MiErrorRuntime(String msg) {
        super(msg);
    }
}
```

Ejemplos de excepciones personalizadas (creadas por nosotros)

✓ Personalizada *controlada* (checked)

```
public class NumeroNegativoException extends Exception {
    public NumeroNegativoException(String msg) {
        super(msg);
    }
}
```

✓ Personalizada *no controlada* (unchecked)

```
public class DatoInvalidoRuntimeException extends RuntimeException {
    public DatoInvalidoRuntimeException(String msg) {
        super(msg);
    }
}
```



LISTAS PEDIDAS

✓ Situaciones donde se prefiere una **excepción controlada** (checked exception)

Estas situaciones ocurren por causas **externas** al programa, y el programador puede intentar recuperarse.

1. Errores de entrada/salida (IO)

Ej.: no se puede leer un archivo (`IOException`).

2. Errores de acceso a bases de datos

Ej.: conexión fallida (`SQLException`).

3. Recursos externos no disponibles

Ej.: un servidor no responde, un fichero no se encuentra.

4. Operaciones que el usuario podría corregir

Ej.: pedir otro archivo, reintentar conexión, etc.

✓ Situaciones donde se prefiere una **excepción no controlada** (unchecked exception)

Son errores **de programación** o **violaciones de precondiciones**.

1. Argumentos inválidos pasados por el programador

Ej.: `IllegalArgumentException`.

2. Acceso a un índice fuera de rango

Ej.: `IndexOutOfBoundsException`.

3. Usar un objeto nulo

Ej.: `NullPointerException`.

4. Errores de programación irreversibles

Ej.: lógica incorrecta, estructuras mal construidas, invariantes rotas.



Resumen perfecto para examen

- **Checked (controladas)** → el compilador obliga a manejarlas; heredan de `Exception` (menos `RuntimeException`).
Son errores externos y recuperables.
- **Unchecked (no controladas)** → no hace falta declararlas ni capturarlas; heredan de `RuntimeException`.
Son errores lógicos del programador y suelen ser irreversibles.
- `RuntimeException` → base de todas las excepciones no controladas.
- Podemos crear nuestras propias excepciones tanto checked como unchecked.

12.1 ¿Qué es `throws`?

`throws` es una **cláusula** que se pone en la **declaración de un método** para indicar que dicho método **puede lanzar** una o varias **excepciones controladas (checked exceptions)**.

Ejemplo:

```
public static double raiz(double x) throws NumeroNegativoException {  
    if (x < 0) {  
        throw new NumeroNegativoException("Número negativo");  
    }  
    return Math.sqrt(x);  
}
```

Con **throws**, el método **avisa** al exterior:

“Ojo: si me llamas, puede que lance esta excepción; tú decides qué hacer con ella”.

12.2 ¿Para qué se usa **throws**?

Para **propagar** una excepción hacia el llamador **sin tener que capturarla** dentro del método.

Sirve cuando:

- el método **no quiere** decidir cómo manejar el error,
- o **no puede** resolverlo,
- o el llamador es quien tiene la información necesaria para manejarlo.

12.3 ¿Por qué **throws** es una alternativa a capturar una excepción controlada?

Porque en Java, para una **excepción controlada**, el compilador exige que el método haga **una de dos cosas**:

✓ Opción 1: Capturarla

Manejar la excepción dentro del método:

```
try {  
    // código  
} catch (IOException e) {  
    // manejar el error aquí  
}
```

✓ Opción 2: Declararla con **throws**

Delegar la responsabilidad del manejo a quien llama:

```
public void leerFichero() throws IOException {  
    // código que puede lanzar IOException  
}
```


👉 **Ambas opciones cumplen con el requisito del compilador.**

Pero capturarla y declararla son *alternativas contrapuestas*:

- Si eliges **capturarla**, la tratas tú.
- Si eliges **lanzarla con throws**, la tratará quien te llame.

🔗 Ejemplo sencillo mostrando la alternativa capturar vs. declarar

✓ Alternativa A — Capturar la excepción

```
public static void abrir() {  
    try {  
        new FileInputStream("datos.txt");  
    } catch (FileNotFoundException e) {  
        System.out.println("Archivo no encontrado.");  
    }  
}
```

✓ Alternativa B — Declarar la excepción con throws

```
public static void abrir() throws FileNotFoundException {  
    new FileInputStream("datos.txt");  
}
```

Ambas son válidas.

La diferencia está en **quién maneja el error**.

📌 Resumen perfecto para examen

- **throws** indica que un método **puede lanzar** ciertas excepciones.
- Se usa para **propagar la excepción hacia arriba** en la pila de llamadas.
- Es una **alternativa obligatoria** a capturar una *excepción controlada*.
- Para una *checked exception*, el compilador exige:
 - ✓ capturarla o
 - ✓ declararla con **throws**.
- Al usar **throws**, el método **no resuelve** el problema: deja la responsabilidad al llamador.

13. Pon un ejemplo en Java de firma de método que incluya **throws**, de una función que abre un fichero pero que declara que no le interesa manejar la excepción de si el fichero no existe, sino que se propague hacia arriba. Eso sí, acuérdate del **finally**.

13. Ejemplo completo en Java

✓ Clase con el método que abre un fichero y declara **throws**

```
import java.io.FileInputStream;
import java.io.FileNotFoundException;
import java.io.IOException;

public class LectorFichero {

    public static void abrirFichero(String nombre) throws FileNotFoundException {
        FileInputStream fis = null;

        try {
            fis = new FileInputStream(nombre);    // Puede lanzar
            FileNotFoundException
            System.out.println("Fichero abierto correctamente.");
        } finally {
            // Se ejecuta SIEMPRE, aunque haya excepción y se propague
            if (fis != null) {
                try {
                    fis.close();
                    System.out.println("Fichero cerrado.");
                } catch (IOException e) {
                    System.out.println("Error al cerrar el fichero.");
                }
            }
        }
    }
}
```

Explicación clara (para examen)

- El método **no captura** `FileNotFoundException`.
- Por tanto, **la firma incluye** `throws FileNotFoundException`.
- Si el fichero no existe, la excepción **se propaga hacia arriba**.
- El bloque **finally se ejecuta siempre**, garantizando el cierre si el fichero llegó a abrirse.
- Aunque la excepción salga del método, el **finally se ejecuta antes** de que la propagación continúe.

✓ Ejemplo de uso desde **main**

```
public class Main {  
    public static void main(String[] args) {  
  
        try {  
            LectorFichero.abrirFichero("datos.txt");  
  
        } catch (FileNotFoundException e) {  
            System.out.println("El fichero no existe: " + e.getMessage());  
        }  
    }  
}
```



Resumen perfecto para examen

- El método **incluye throws** para delegar el manejo al exterior.
- No usa **catch** porque **no quiere manejar** la excepción.
- El bloque **finally siempre se ejecuta**, asegurando la liberación de recursos.
- Si el fichero no existe → la excepción se propaga y será tratada en el **main**.

☒ 14.1. ¿Podemos poner en **throws** excepciones no controladas, como **RuntimeException**?

Sí, se puede.

Nada en Java lo prohíbe.

Ejemplo perfectamente válido:

```
public void calcular() throws RuntimeException {  
    // ...  
}
```

Pero...

🔗 **No sirve de mucho**, porque el compilador **no obliga** a declararlas en **throws**.

☒ 14.2. ¿Debe el método llamador poner **try-catch** si aparece una **RuntimeException** en **throws**?

No, no está obligado.

Incluso si el método declara:

```
public void algo() throws NullPointerException
```

El llamador **no** tiene que capturarla:

```
algo();    // válido sin try-catch
```

Recuerda:

- Las excepciones **unchecked** (las que heredan de `RuntimeException`)
 - **no es obligatorio declararlas**
 - **no es obligatorio capturarlas**

Por eso el `throws` en este caso es **puramente informativo**.

☑ 14.3. ¿Qué sentido tendría poner una excepción no controlada en `throws`?

Aunque no sea obligatorio, **puede tener sentido en algunos casos**, especialmente para documentación del contrato del método:

✓ 1. Documentar el comportamiento esperado del método

Indica al lector:

```
"Este método puede lanzar esta excepción si le pasas argumentos inválidos".
```

Ejemplo:

```
public int dividir(int a, int b) throws ArithmeticException {  
    return a / b;  
}
```

✓ 2. Hacer más explícita la API

Ayuda a los usuarios de tu clase a saber que deben cumplir ciertas precondiciones.

✓ 3. Mejorar herramientas automáticas de documentación

El `throws` aparece en el Javadoc e informa del uso correcto del método.

✓ 4. Estandarizar el estilo del equipo

Algunos equipos prefieren declarar incluso las excepciones no controladas para mayor claridad.



Resumen perfecto para examen

- ✓ **Sí**, se pueden poner excepciones no controladas (`RuntimeException`) en `throws`.
- ✓ **No**, el método llamador **no está obligado** a capturarlas.
- ✓ El `throws` sobre unchecked exceptions **solo sirve como documentación o claridad**.
- ✓ El compilador **no exige** ni declararlas ni capturarlas.
- ✓ Se usa cuando queremos que la API sea más explícita sobre precondiciones.

15.1 ☒ ¿Cuándo se recomienda usar **excepciones controladas** (checked)?

Las excepciones **controladas** (como `IOException`, `SQLException`, `FileNotFoundException`) se recomiendan cuando:

✓ 1. El error **proviene del exterior** del programa

Ejemplos:

- Fichero no encontrado
- Error de red
- Error de base de datos
- Problemas al leer/escribir en disco

El programa **no controla** estas condiciones: dependen del SO, del usuario, de la red...

✓ 2. Son errores **previsibles y recuperables**

El llamador puede:

- reintentar
- pedir otro archivo
- mostrar un mensaje al usuario
- buscar una alternativa

✓ 3. Es importante **forzar al programador** a manejar el error

El compilador obliga a escribir:

- un `try-catch`, o
- un `throws`,

para evitar que el error se ignore.



En resumen:

Se usan para errores externos, previsibles y recuperables.

15.2 ¿Cuándo se recomiendan usar **excepciones no controladas** (unchecked)?

Son las que heredan de `RuntimeException` (`IllegalArgumentException`, `NullPointerException`, `IndexOutOfBoundsException`...).

Se usan cuando el error:

- ✓ 1. Es causado por un **fallo del programador**

Ejemplos:

- Pasar un argumento inválido → `IllegalArgumentException`
- Acceder a un índice fuera de rango → `IndexOutOfBoundsException`
- Llamar a un método con un objeto nulo → `NullPointerException`

Estos errores **no son recuperables** programáticamente:
la solución es **corregir el código**, no manejar la excepción.

- ✓ 2. No tendría sentido obligar a capturarlas

Forzar `try-catch` en cada operación que pueda dar `NullPointerException` sería absurdo.

- ✓ 3. Son violaciones de precondiciones

El método lanza una excepción no controlada para indicar:

“el método se está usando mal”.

 En resumen:

Se usan para errores de programación, inesperados o irrecuperables.

Resumen claro (examen-ready)

Tipo	Ejemplos	Cuándo usarlas
Controladas (checked)	<code>IOException</code> , <code>SQLException</code>	Errores externos, previsibles y recuperables.
No controladas (unchecked)	<code>IllegalArgumentException</code> , <code>NullPointerException</code>	Fallos de programación, violación de precondiciones, errores no recuperables.

15.3 ☒ ¿Existen ambas opciones en todos los lenguajes?

No.

El modelo de excepciones controladas (checked) de Java es **muy poco común** en otros lenguajes.

Lenguajes que **NO tienen excepciones controladas**:

- **C++**
- **Python**
- **C#**
- **JavaScript**
- **Go** (usa valores de retorno, no excepciones)
- **Rust** (usa **Result**, no excepciones)

Estos lenguajes solo tienen **excepciones no controladas**, o directamente otro sistema (Go/Rust).

15.4 ☒ En los lenguajes donde solo existe una opción, ¿cuál es la más habitual?

En casi todos los lenguajes modernos, la opción habitual es:

🔗 **Excepciones no controladas (estilo RuntimeException)**

Es decir:

- Puedes lanzar una excepción.
- Puedes capturarla si quieres.
- El compilador **no te obliga** ni a declararla ni a capturarla.

Este modelo es más simple y es el que sigue prácticamente todo el mundo *excepto Java*.

Resumen final perfecto para examen

- **Controladas (checked)** → se usan para errores externos y recuperables; el compilador obliga a manejarlas.
 - **No controladas (unchecked)** → se usan para errores de programación o violación de precondiciones.
 - **RuntimeException** es la base de todas las no controladas.
 - **La mayoría de lenguajes no tienen checked exceptions.**
 - Donde solo existe una opción, la habitual es el modelo **no controlado**, como Python, C#, C++, JavaScript.
-
-

☑ 16.1. ¿Tiene sentido lanzar excepciones *dentro* de un `catch`?

Sí.

Tiene sentido cuando, al capturar una excepción, descubres que:

- no puedes resolverla,
- o necesitas convertirla en una excepción diferente,
- o necesitas añadir más información,
- o el nivel actual no es el adecuado para manejarla.

En esos casos, puedes lanzar **una nueva excepción** desde el `catch`.

✓ Ejemplo: convertir una excepción en otra

```
try {
    FileInputStream f = new FileInputStream("datos.txt");
} catch (FileNotFoundException e) {
    throw new IllegalStateException("No se pudo abrir el archivo requerido", e);
}
```

Aquí se captura la excepción original pero se lanza **otra**, más adecuada para el contexto.

☑ 16.2. ¿Se puede relanzar la **misma** excepción capturada?

Sí, perfectamente.

Es tan simple como:

```
catch (Exception e) {
    throw e;
}
```

? 16.3. ¿Cuándo tiene sentido *relanzar* la misma excepción?

Tiene sentido cuando:

✓ 1. El método quiere **registrar, imprimir o limpiar recursos**, pero **no manejar** realmente la excepción

Por ejemplo:

- escribir un log,
- cerrar recursos,
- notificar algo...

y después dejar que la excepción siga propagándose.

- ✓ 2. El método está en una capa intermedia que **no debe decidir cómo manejar** el error

El llamador (en una capa superior) es quien tiene la responsabilidad final.

- ✓ 3. Se quiere *añadir acciones adicionales* antes de que la excepción siga su camino

🔗 Ejemplo: relanzar la misma excepción capturada

```
public static void procesar() throws IOException {
    FileInputStream fis = null;

    try {
        fis = new FileInputStream("datos.txt");
        // procesar datos...

    } catch (IOException e) {
        System.out.println("Log: error al abrir archivo.");
        throw e;    // ← se relanza la MISMA excepción
    } finally {
        if (fis != null) fis.close();
    }
}
```

Aquí:

- Se escribe un mensaje de log.
- Se relanza la excepción original.
- El llamador decidirá qué hacer.

🔗 Ejemplo: lanzar otra excepción distinta dentro del **catch**

```
public static void cargarConfiguracion() throws ConfigException {
    try {
        new FileInputStream("config.txt");
    } catch (FileNotFoundException e) {
        throw new ConfigException("Falta el archivo de configuración", e);
    }
}
```

Esto se llama **encapsular** la excepción en un tipo más significativo para la aplicación.

Resumen perfecto para examen

- ✓ **Sí**, se pueden lanzar excepciones dentro de un **catch**.
 - ✓ **Sí**, se puede relanzar la **misma** excepción capturada.
 - ✓ Lanzar una excepción en el **catch** se usa para:
 - convertir la excepción en otra,
 - añadir información,
 - dejar que otro nivel la maneje.
 - ✓ Relanzar la misma se usa cuando quieres:
 - registrar/loguear el error sin manejarlo,
 - ejecutar acciones antes de la propagación,
 - dejar que otro método decida cómo actuar.
-

17.1 ¿Qué significa que una excepción sea la “**causa**” de otra?

En Java, una excepción puede llevar dentro otra excepción que **originó el problema**. A esto se le llama **exception chaining** (encadenamiento de excepciones).

La excepción de alto nivel:

- **describe el problema en un contexto más general**
- pero **conserva la excepción original** como *causa*

Esto permite que, al depurar, se pueda ver **qué pasó realmente y dónde empezó el error**.

17.2 ¿Por qué es útil?

Porque:

- capas superiores (lógica de negocio) no deberían lanzar excepciones técnicas como **IOException**;
- pero tampoco queremos **perder información** de la causa real del fallo.

Así que la excepción de bajo nivel se **encapsula** dentro de la excepción de más alto nivel.

```
throw new MiExcepcionDeAltoNivel("Fallo cargando configuracion", e);
```

El **e** es la **causa**.

17.3 ¿Se ve la causa al imprimir la excepción?

Sí.

Cuando una excepción tiene causa, el *stack trace* que se imprime en pantalla muestra:

1. la excepción de alto nivel
2. y debajo: Caused by: <excepción original>

Esto ayuda mucho al depurar.

Ejemplo completo en Java

Encapsular una excepción de bajo nivel (**FileNotFoundException**) en una personalizada de alto nivel.

1. Excepción personalizada de alto nivel

```
public class ConfigException extends Exception {  
    public ConfigException(String mensaje, Throwable causa) {  
        super(mensaje, causa);  
    }  
}
```

2. Método que captura una excepción técnica y la encapsula

```
import java.io.FileInputStream;  
import java.io.FileNotFoundException;  
  
public class LectorConfiguracion {  
  
    public static void cargar() throws ConfigException {  
  
        try {  
            // Excepción de bajo nivel  
            FileInputStream fis = new FileInputStream("config.txt");  
  
        } catch (FileNotFoundException e) {  
            // Encapsulamos la excepción original como "causa"  
            throw new ConfigException("No se pudo cargar la configuración", e);  
        }  
    }  
}
```

3. Llamador que deja ver la traza

```
public class Main {  
    public static void main(String[] args) {  
        try {  
            LectorConfiguracion.cargar();  
  
        } catch (ConfigException e) {  
            e.printStackTrace(); // ← Veremos también la causa  
        }  
    }  
}
```

17.4 ¿Qué veríamos en pantalla?

Algo así:

```
ConfigException: No se pudo cargar la configuración  
    at LectorConfiguracion.cargar(LectorConfiguracion.java:12)  
    at Main.main(Main.java:6)  
Caused by: java.io.FileNotFoundException: config.txt (No existe el fichero)  
    at java.io.FileInputStream.<init>(FileInputStream.java:123)  
    ... 2 more
```

Observa el:

```
Caused by: FileNotFoundException
```

Eso muestra **la causa real del error**, tal como pide la pregunta.



Resumen perfecto para examen

- Una excepción puede tener otra excepción como **causa**.
- Esto sirve para:
 - ✓ transformar errores de bajo nivel en errores de alto nivel,
 - ✓ sin perder la información original del fallo.
- Sí, se puede ver la causa al imprimir la excepción; aparece como:
“**Caused by: ...**”
- Se usa la técnica del **encadenamiento de excepciones**:

```
throw new MiExcepcion("mensaje", causa);
```
